Prvi praktični test iz
Algoritama i struktura podataka

Prvi praktični test održan 25.09.2025.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaG_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Quick Sort*

Napisati ***quick*** metodu ***Quick Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalaze klase ***RandomArrayGenerator*** i ***RandomNumberGen*** koje treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***QuickSort*** napisati implementaciju za metodu ***private static void quickSort(int[] arr, int lower, int upper)***
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – Promena redosleda poslednjih n elemenata steka koristeći red

Implementirati funkciju koja za zadati **stek** celih brojeva obrće redosled samo poslednjih n elemenata steka, koristeći pomoćni **red**. U okviru tih poslednjih n elemenata, za svaki od njih koji je **neparan** važiće da se njegova vrednost umanji za 2, a za svaki od tih n elemenata koji je paran njegova vrednost će se udvostručiti. Ukoliko nakon umanjivanja neparnog broja za 2 dobijemo negativan broj, vrednost tog broja se postavlja na 1. Osim toga, ostali elementi steka ostaju u istom redosledu.

Specifikacija funkcije:

- Funkcija prima broj n, gde je n broj elemenata koje treba obrnuti.
- Funkcija treba koristiti red (***queue***) kao pomoćnu strukturu.
- Elementi pre prosleđenih n treba da ostanu u originalnom redosledu.

Dodatni zahtevi za rešavanje zadatka:

- Algoritam pisati u okviru klase ***Stack.***
- Zahtevan potpis metode: ***public void ReverseAndModifyLastNElements (int n)***

Primer1:

- Originalni stek: 4 3 1 10 9 4, n = 3

  Među poslednja 3 elementa, 10 i 4 su parani brojevi čija vrednost će se udvostručiti, dok je broj 9 neparan broj i njegova vrednost se smanjuje za 2.

- Stek nakon obrtanja 3 elementa: 4 3 1 8 7 20

Primer2:

- Originalni stek: 4 3 1 1 2 3 , n=3

Stek nakon obrtanja poslednja 3 elementa: 4 3 1 1 4 1

Primer3:

- Originalni stek: 4 3 1 4 3 2, n = 8

  Nevažeća vrednost za n.

Zadatak 3 – *Broj i proizvod "uvala" i "bregova" u jednostruko povezanoj listi*

U jednostruko spregnutoj listi svaki od čvorova može da predstavlja uvala ili breg.

Čvor će predstavljati **uvala** ukoliko je njegova vrednost manja od oba susedna elementa (element pre i posle posmatranog čvora u okviru liste) – dodatno, razlika u odnosu na oba suseda mora biti najmanje 2.

Čvor će predstavljati **breg** ukoliko je njegova vrednost veća od oba susedna elementa (element pre I posle posmatranog čvora u okviru liste) – dodatno, razlika u odnosu na oba suseda mora biti najmanje 3.

Metoda vraća niz u kome prva vrednost predstavlja broj uvala i bregova, a druga proizvod njihovih vrednosti.

Napomena: čvor može biti uvala ili breg samo ako ima prethodni i sledeći čvor.

Dodatni zahtevi:

- Rešavaj ovaj zadatak pomoću strukture liste (***LinkedList***) iz šablona zadatka.
- Implementiraj ga u okviru metode: ***public int[] CountAndMultiplyValleysAndPeaks()***

<u>Primer1:</u>

- Input 1: lista: 1 -> 3 -> 1 -> 4 -> 2 ->7-> 3 -> 6
- Output 1: [4, 42]

Objašnjenje:

1 je manje za 2 ili više od susednih 3 i 4.

2 je manje za 2 ili više od susednih 4 i 7.

3 je manje za 2 ili više od susednih 7 i 6.

7 je veće za 3 ili više od susednih 2 i 3.

Kritične tačke su:

Uvale: 1, 2 i 3 (1 je manje za 2 ili više od susednih 3 i 4).

Bregovi: 7

Ukupan broj je 4, a proizvod 1*2*3*7 je 42.

Primer2:

- Input 2: lista: 1-> 2 -> 2 -> 5 -> 5
- Output 2: [-1, -1]

Objašnjenje:

Nemamo kritične tačke, odnosno uvale i bregove jer nijedan od čvorova ne zadovoljava postavljene uslove.